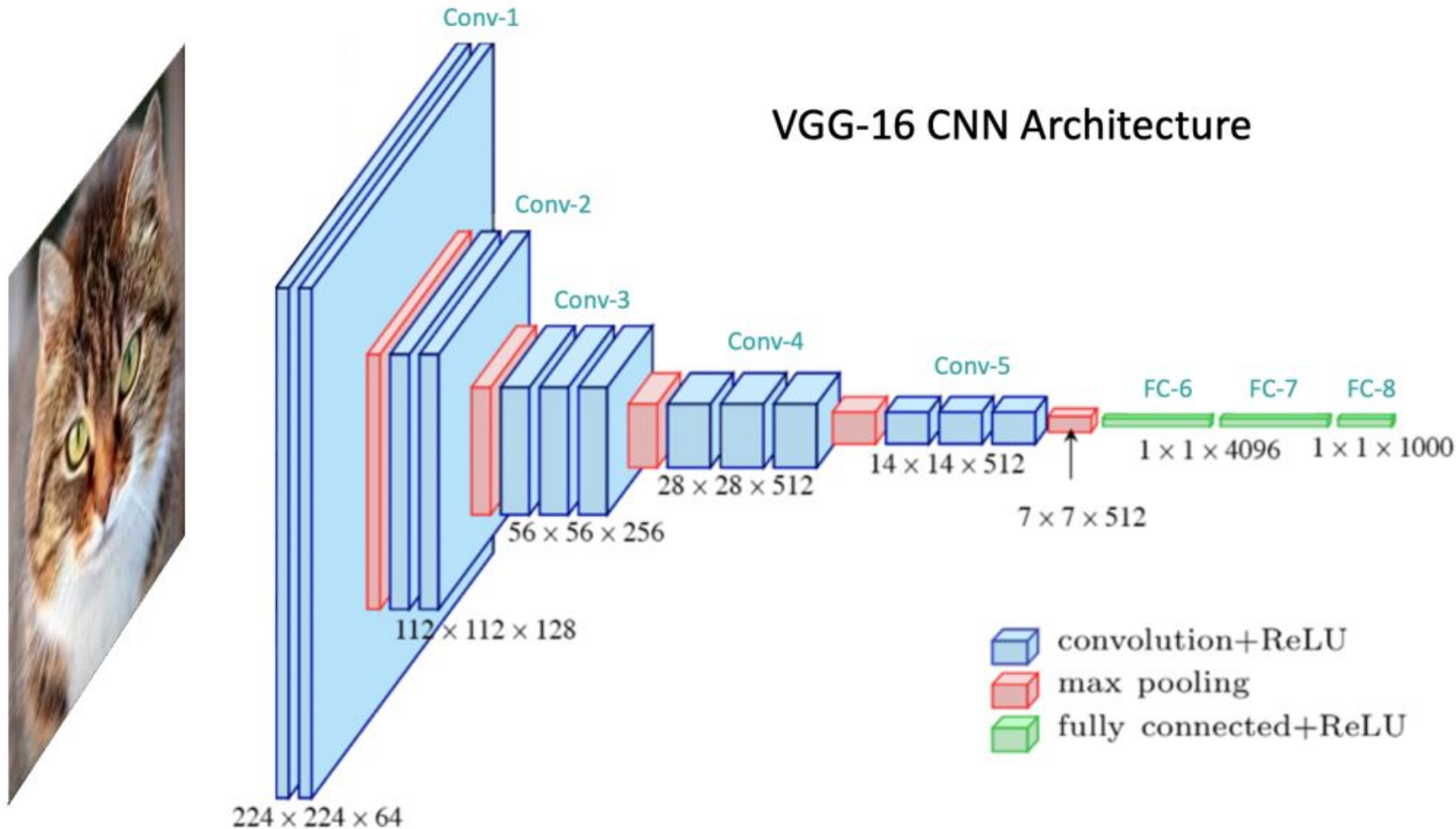


Introduction to Convolutional Neural Networks

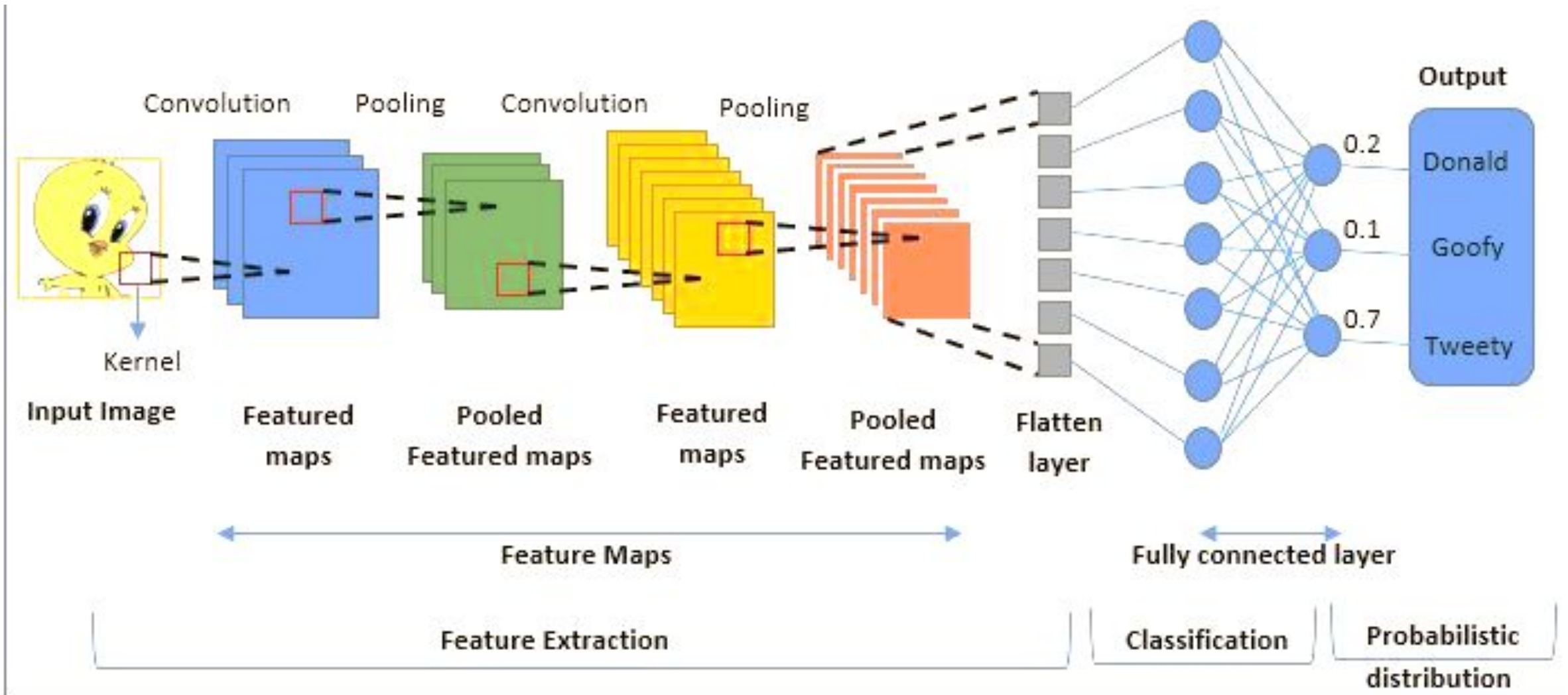
What are CNNs ?

- CNN = Neural Network with a **convolution operation**
- Convolutional neural networks (**CNN, ConvNet**) is a class of **deep, feed-forward artificial neural networks** that are applied to **analyzing visual imagery**.
- Consists of **Matrix Multiplication** in at least one of the layers

Convolutional neural networks

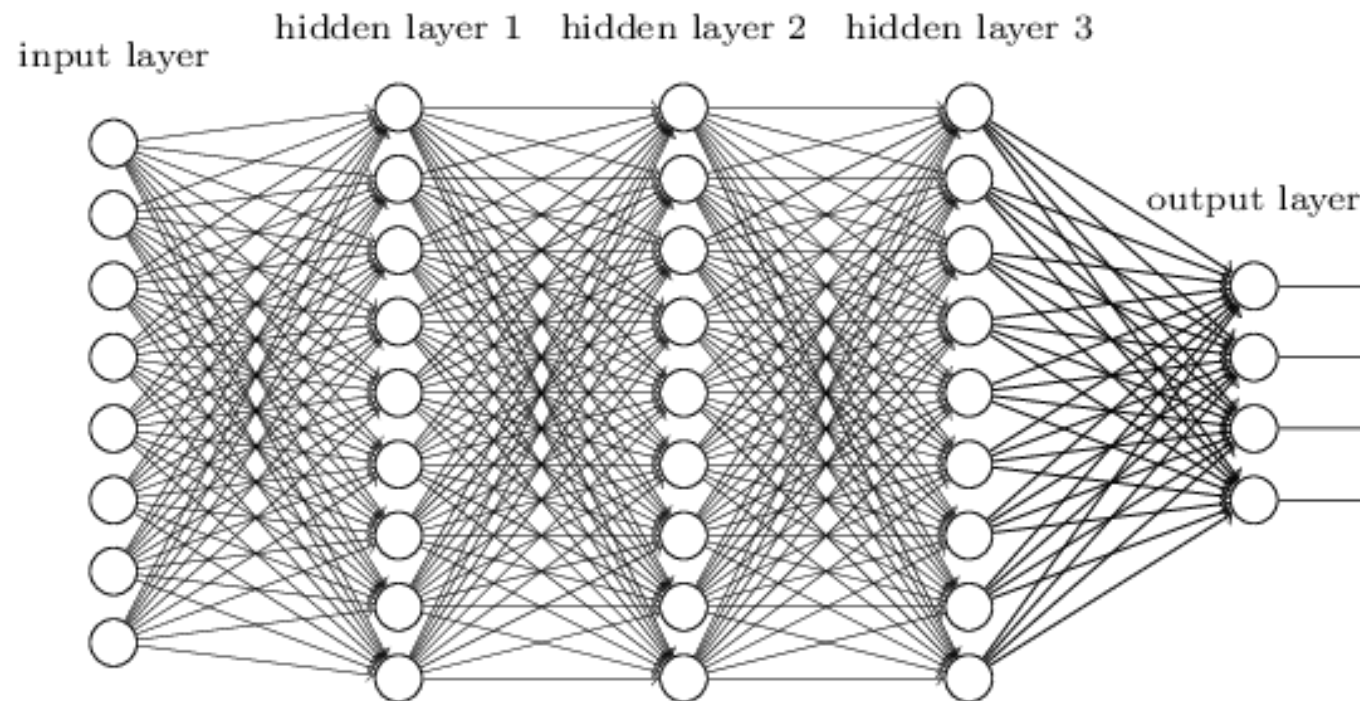


Convolutional neural networks



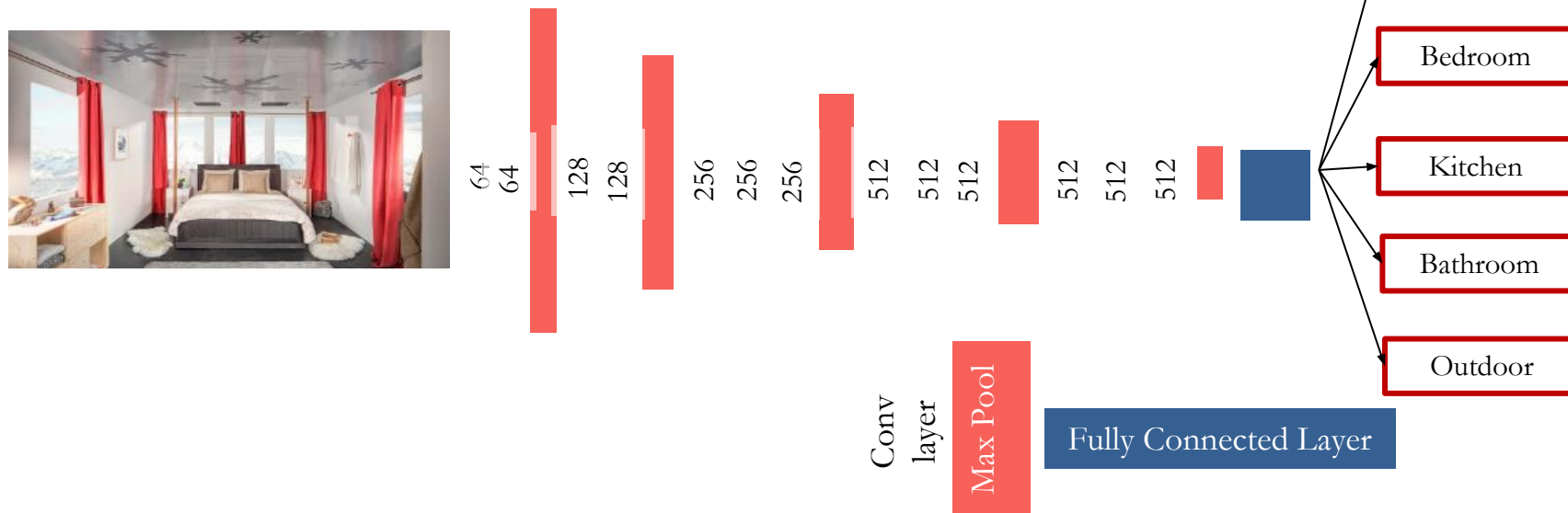
Smaller Network: CNN

- We know it is good to learn a small model.
- From this fully connected model, do we really **need all the edges**?
- **Can some of these be shared?**

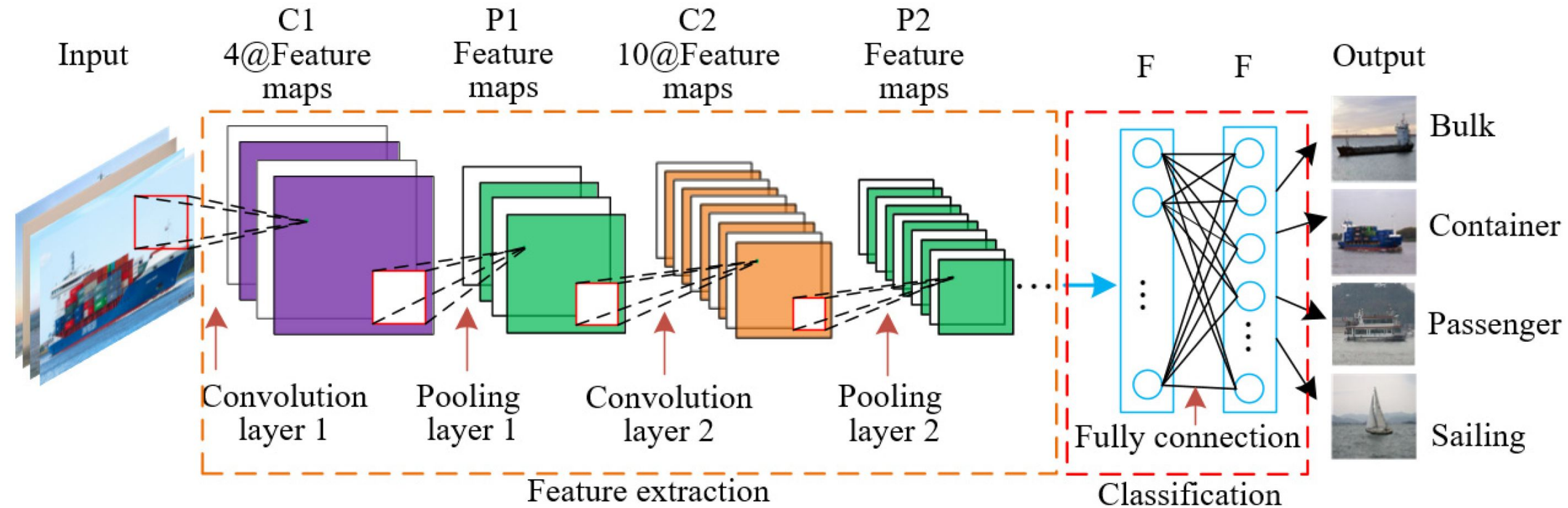


Convolutional Neural Networks (CNNs)

- Feature extraction architecture
 - After **2 convolutional layers**, a **max-pooling** layer reduces the size of the feature maps (typically by 2)
 - A **fully convolutional and a softmax layers** are added last to perform classification



CNN architecture

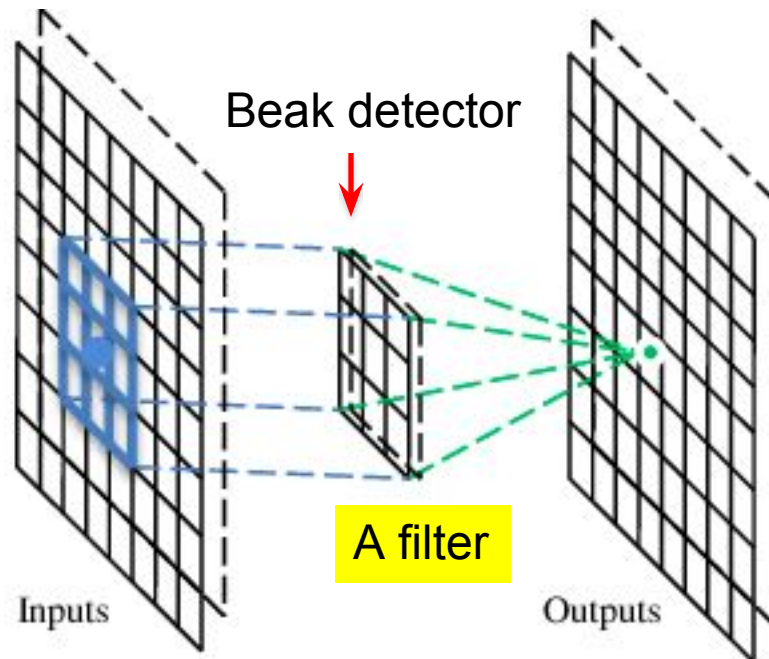


Convolutional Neural Networks (CovNet)

- In a nutshell,
A ConvNet usually has 3 types of layers:
 - 1) **Convolutional Layer** (*CONV*)
 - 2) **Pooling Layer** (*POOL*)
 - 3) **Fully Connected Layer** (*FC*)

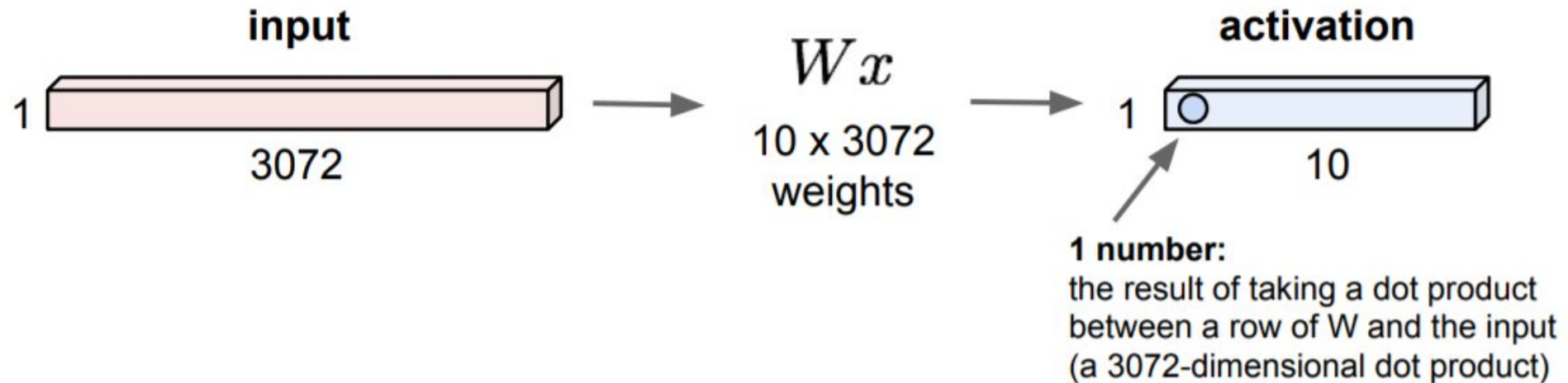
A convolutional layer

A CNN is a neural network with some convolutional layers (and some other layers). A convolutional layer has a number of filters that does convolutional operation.

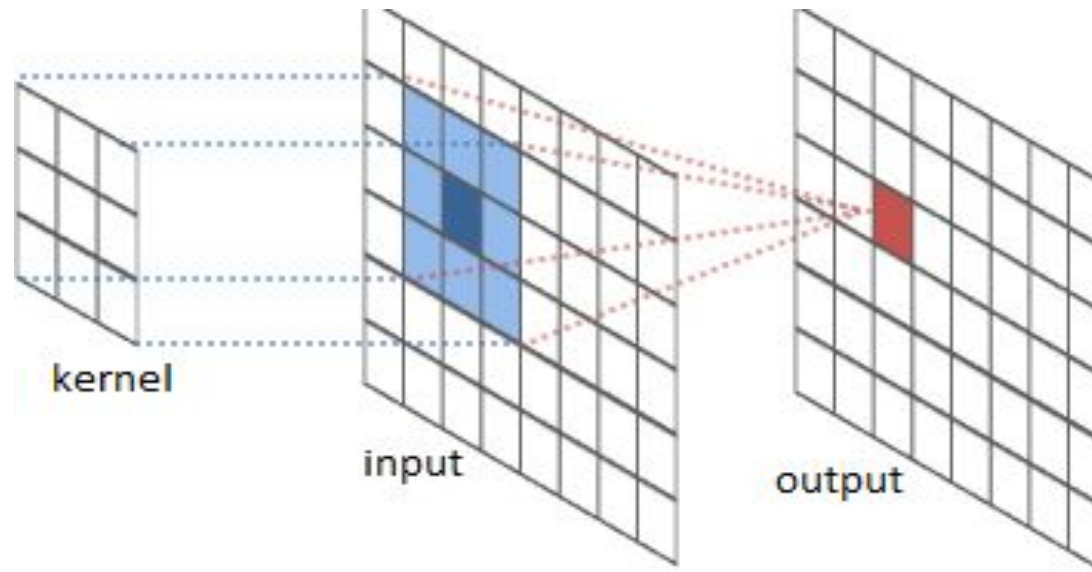


Fully Connected Layer

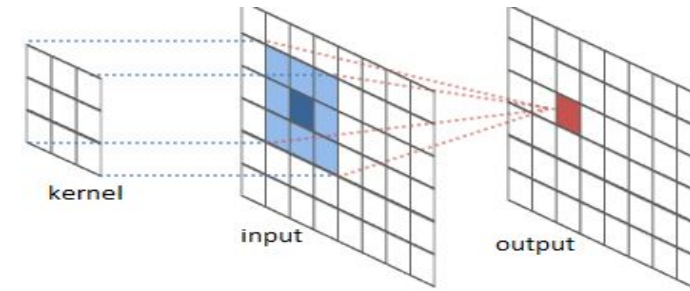
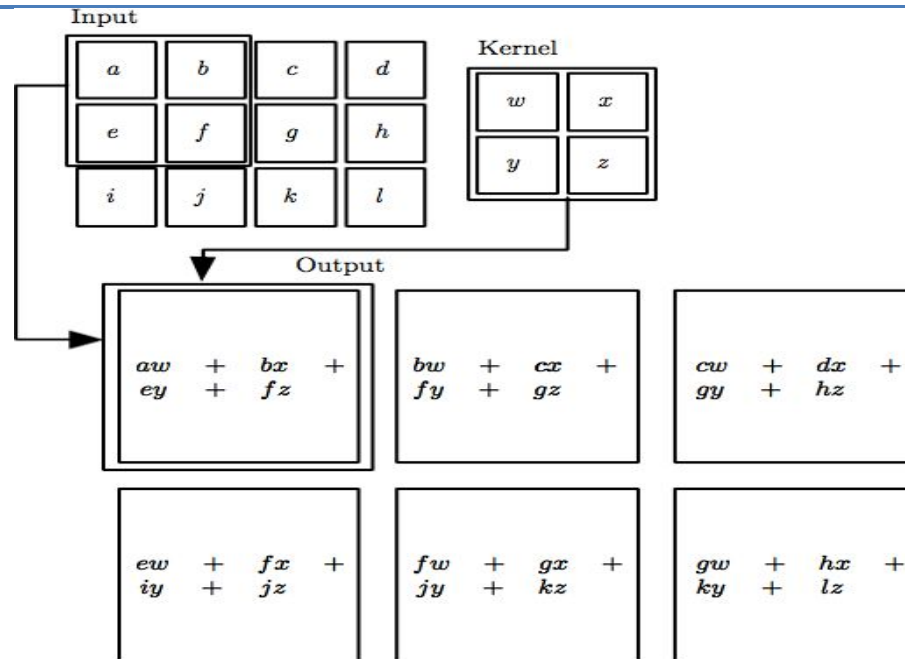
32x32x3 image -> stretch to 3072 x 1



The convolution operation



The convolution operation



Local Connectivity. Neurons will connect to only a local region of the input volume.

The spatial extent of this connectivity is a hyperparameter called the **receptive field** of the neuron (equivalently this is the **filter size**).

Convolution

These are the network parameters to be learned.

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

-1	1	-1
-1	1	-1
-1	1	-1

Filter 2

⋮ ⋮

Each filter detects a small pattern (3 x 3).

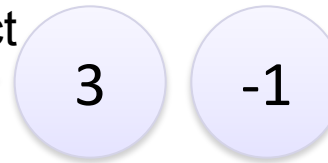
Convolution

stride=1

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

Dot
product



1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

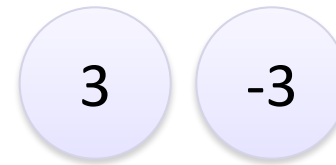
Convolutio

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

If stride=2

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0



6 x 6 image

Convolution

stride=1

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

3	-1	-3	-1
-3	1	0	-3
-3	-3	0	1
3	-2	-2	-1

Filter 1

1	-1	-1
-1	1	-1
-1	-1	1

Convolution

-1	1	-1
-1	1	-1
-1	1	-1

Filter 2

stride=1

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

Repeat this for each filter

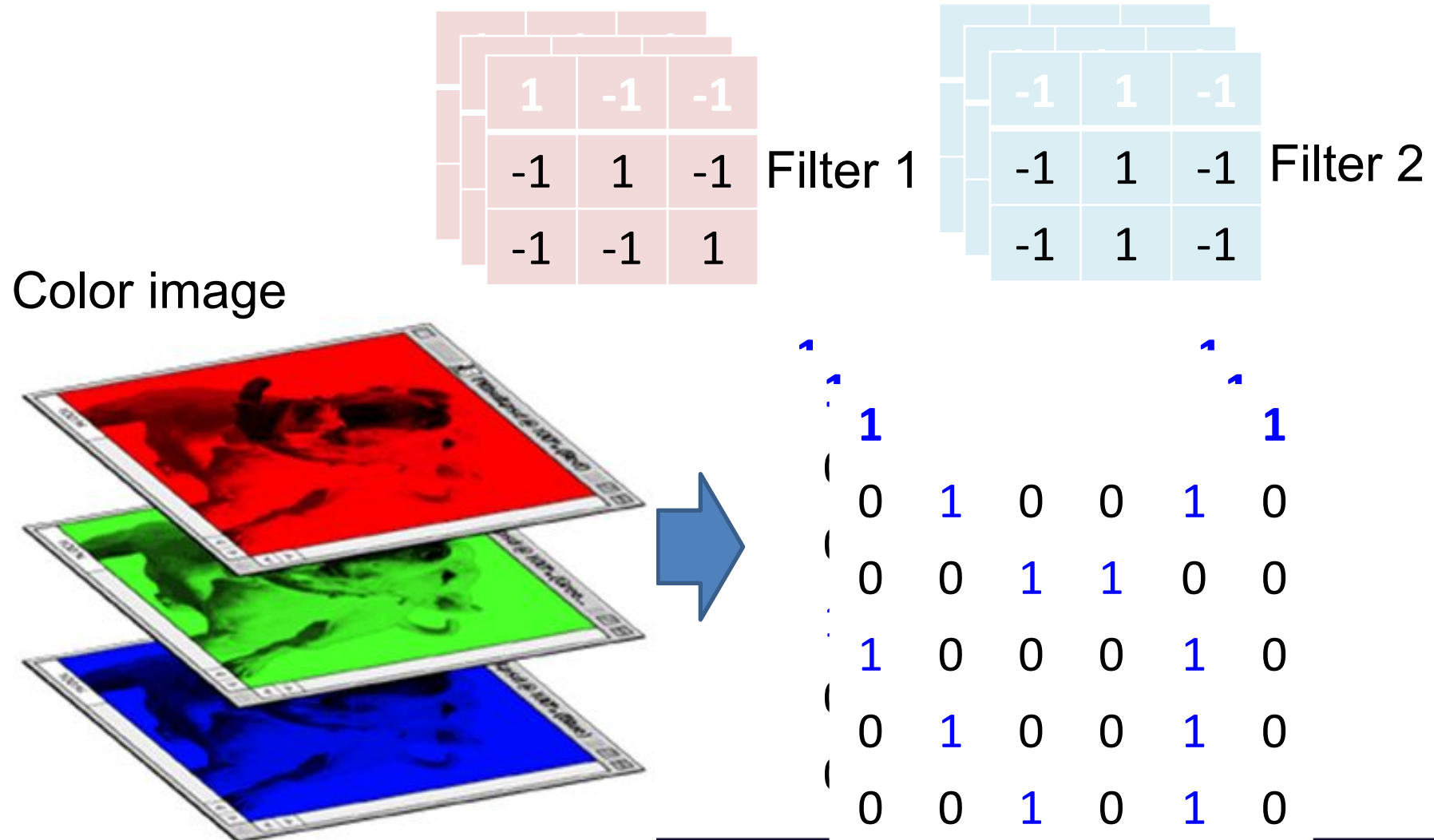
-1	-1	-1	-1
-1	-1	-2	1
-1	-1	-2	1
-1	0	-4	3

Feature Map

Two 4 x 4 images

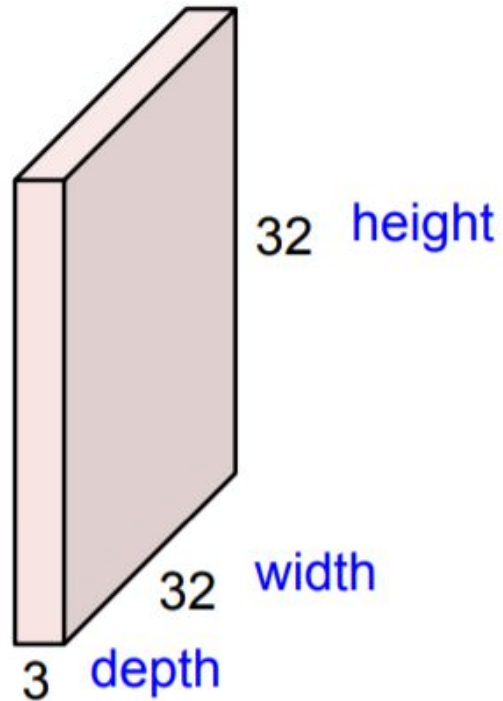
Forming 2 x 4 x 4 matrix

Color image: RGB 3 channels



Convolution Layer

32x32x3 image -> preserve spatial structure

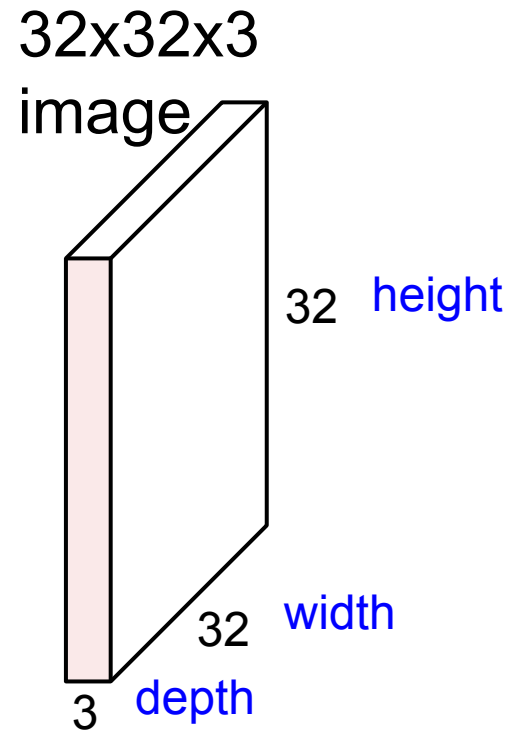


5x5x3 filter



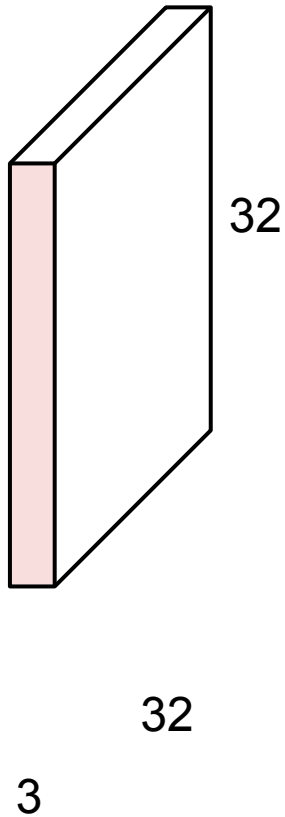
Convolve the filter with the image
i.e. “slide over the image spatially,
computing dot products”

Convolution Layer



Convolution Layer

32x32x3 image



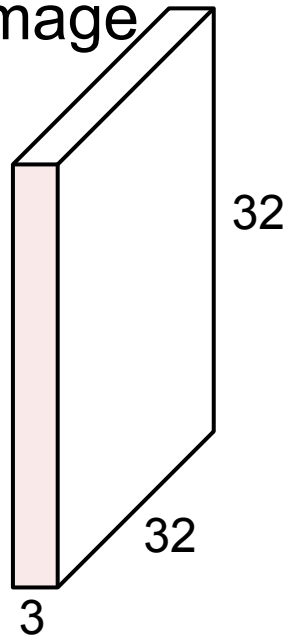
5x5x3 filter



Convolve the filter with the image
i.e. “slide over the image
spatially, computing dot
products”

Convolution Layer

32x32x3
image



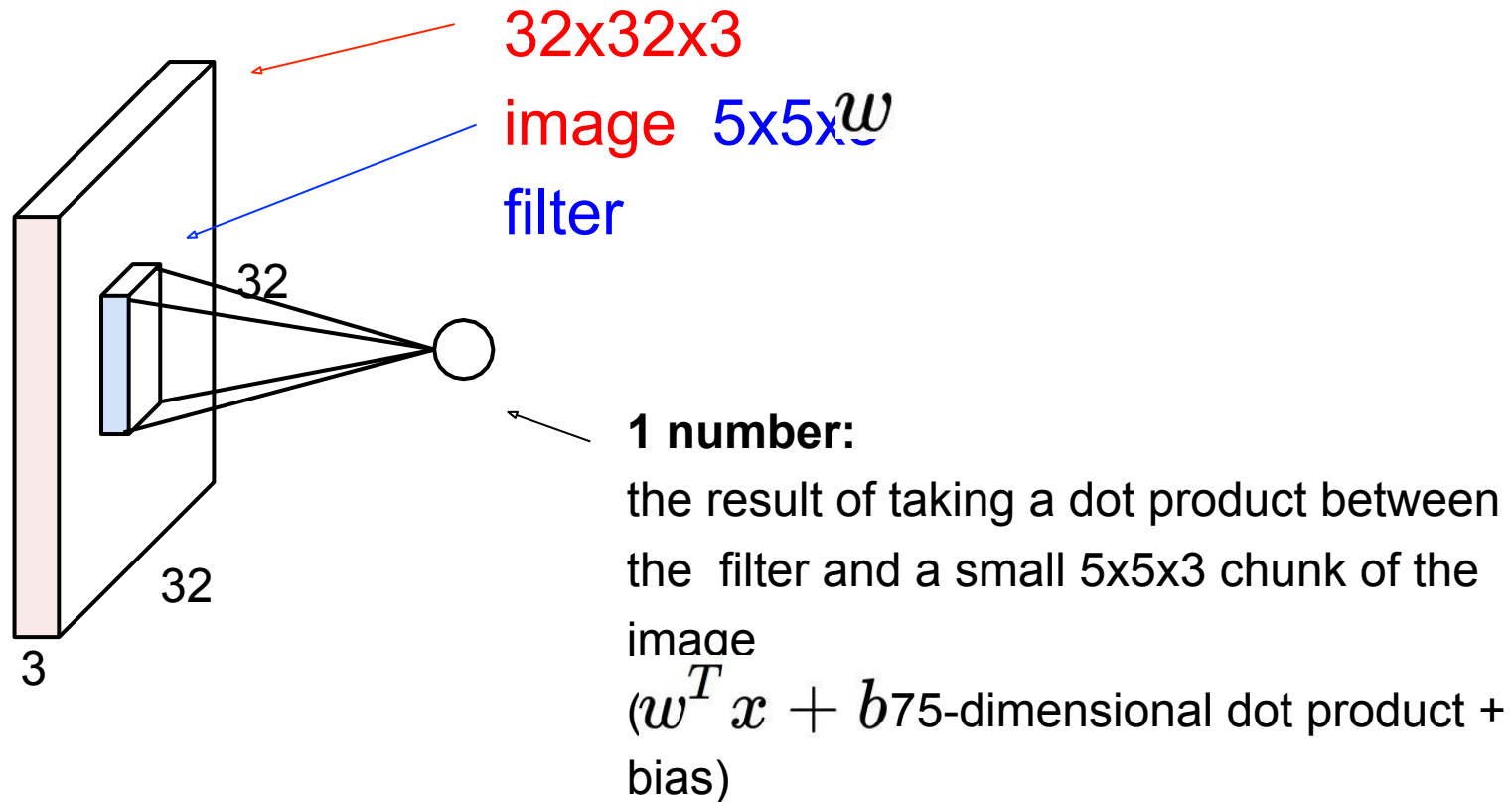
Filters always extend the
depth of the input
volume

5x5x3
filter

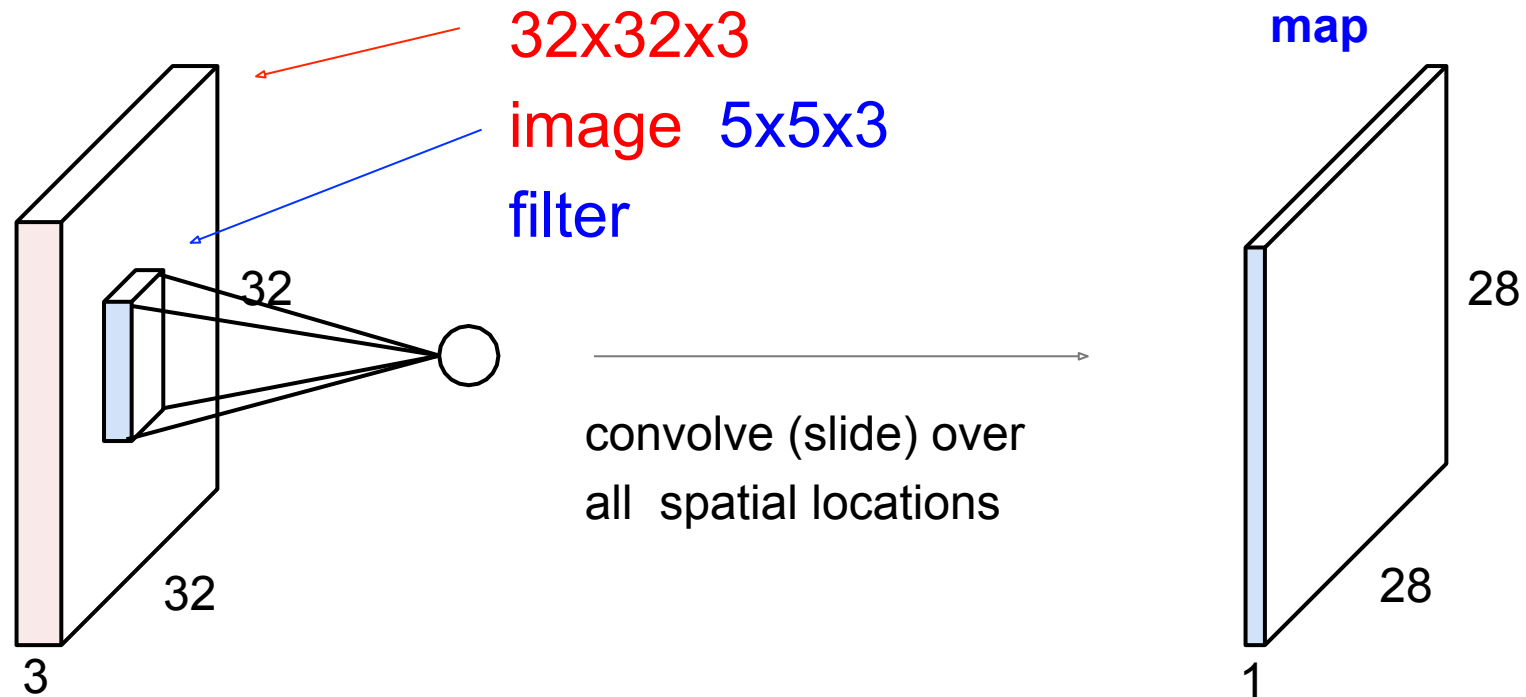


Convolve the filter with the
image
i.e. “slide over the image
spatially, computing dot
products”

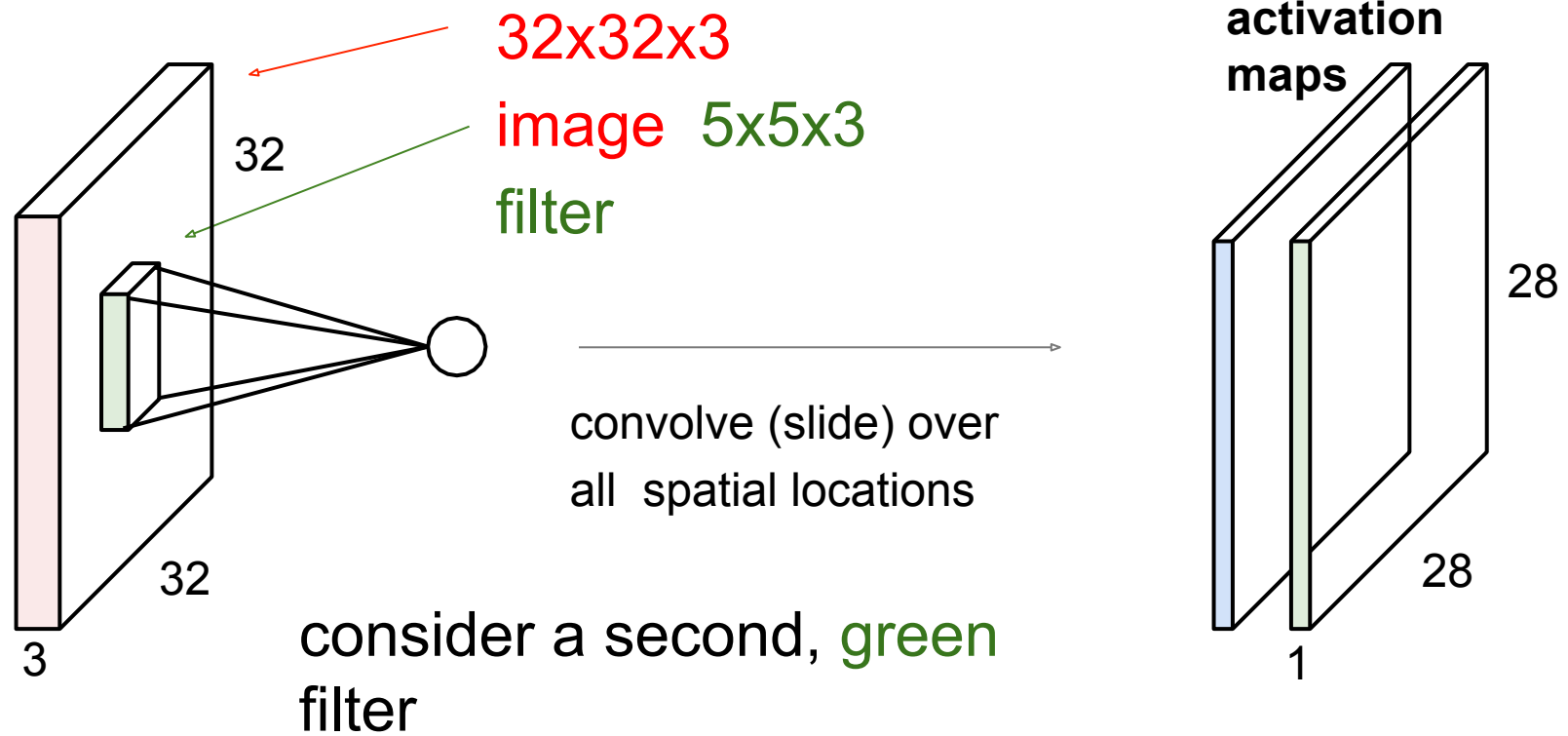
Convolution Layer



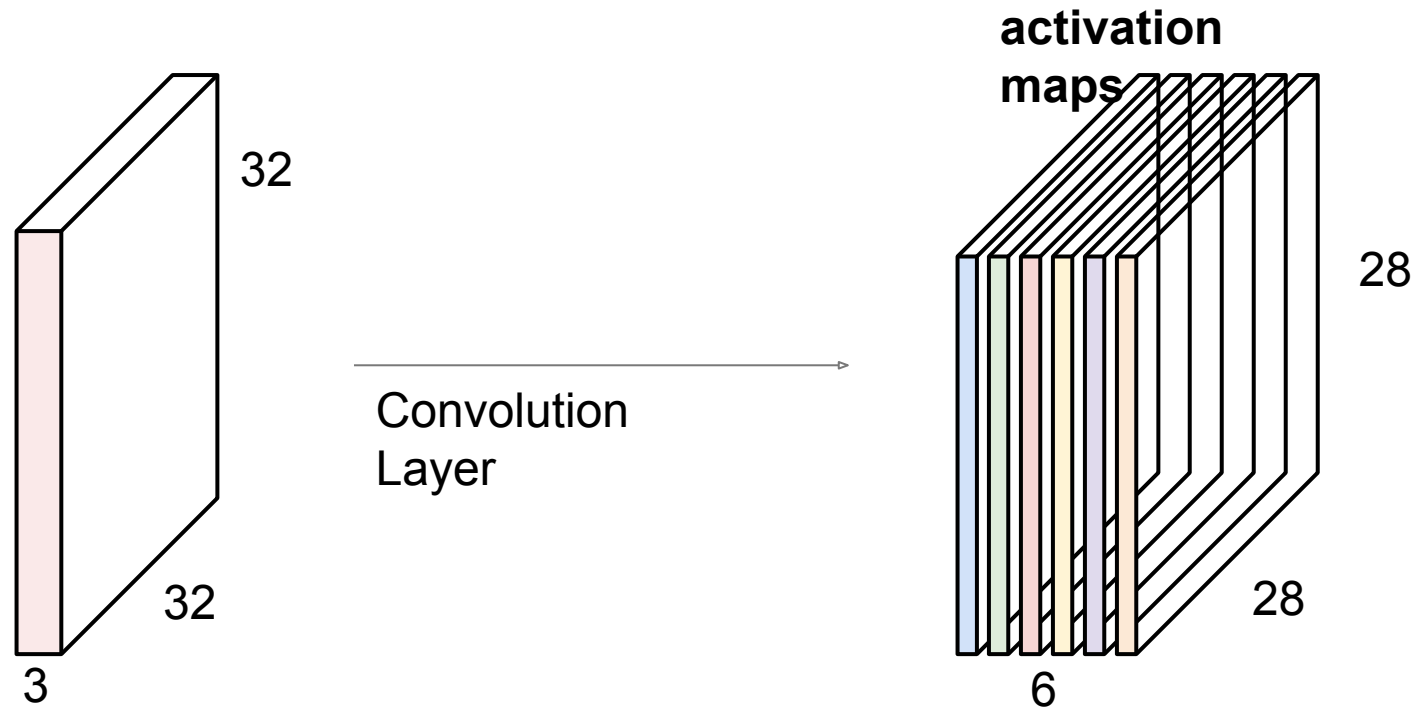
Convolution Layer



Convolution Layer

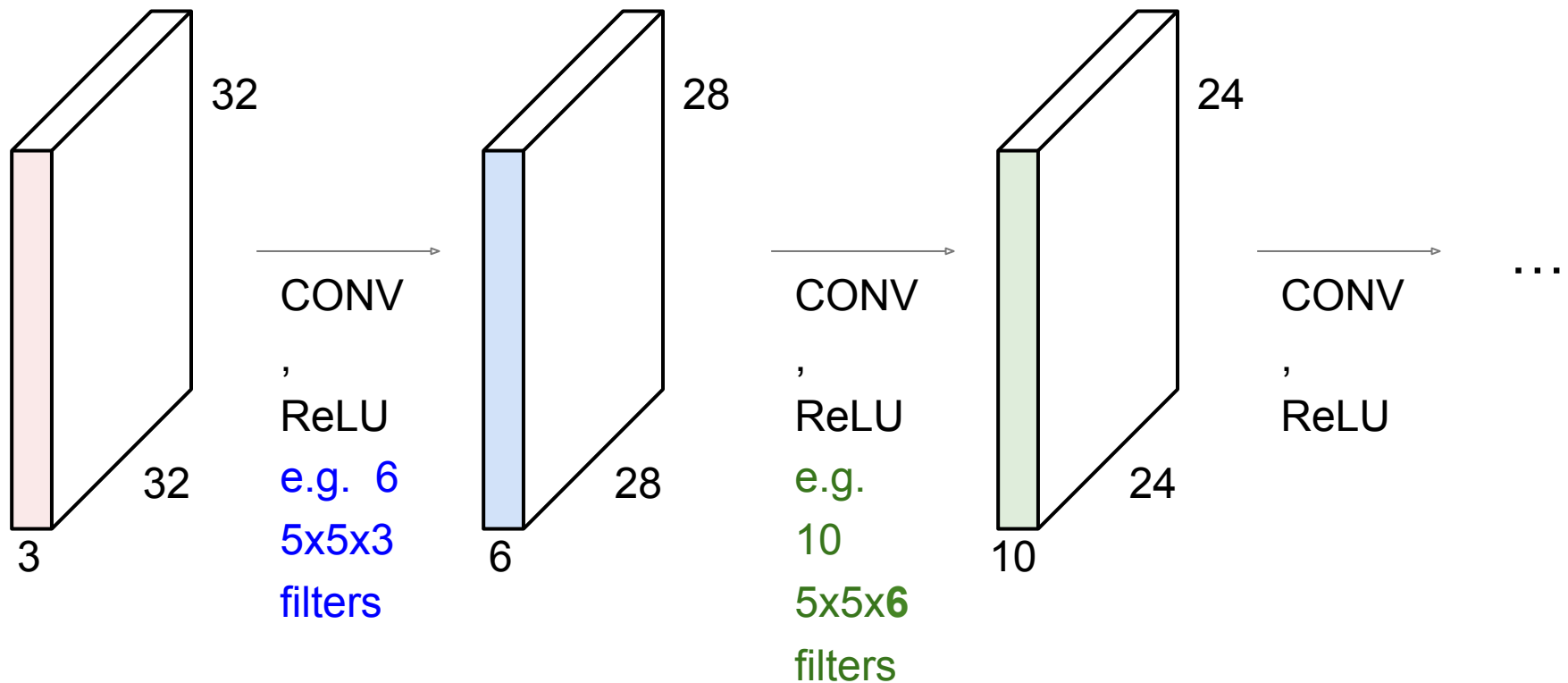


For example, if we had 6 5x5 filters, we'll get 6 separate activation maps:

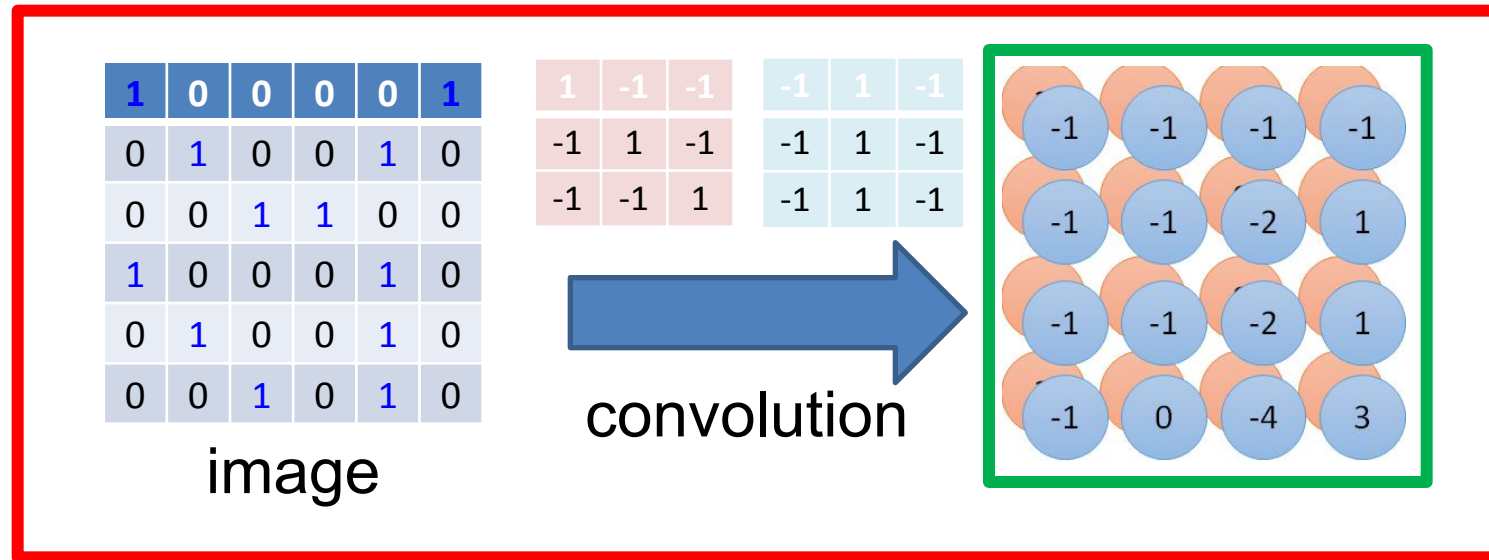


We stack these up to get a “new image” of size 28x28x6!

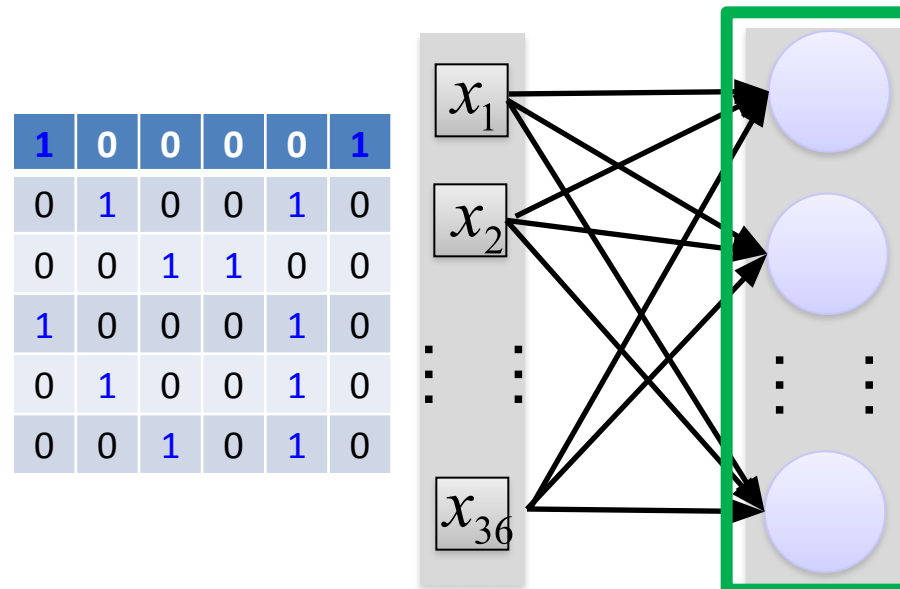
Preview: ConvNet is a sequence of Convolutional Layers, interspersed with activation functions

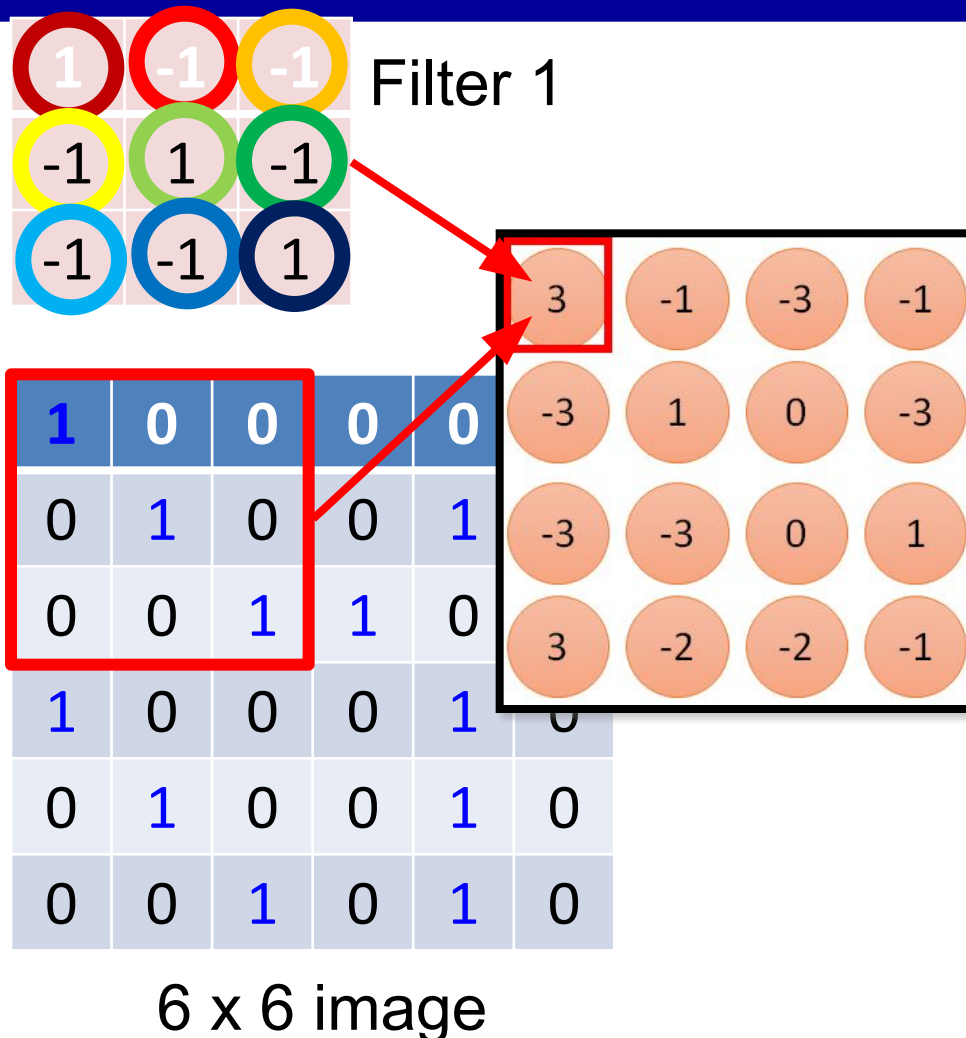


Convolution v.s. Fully Connected

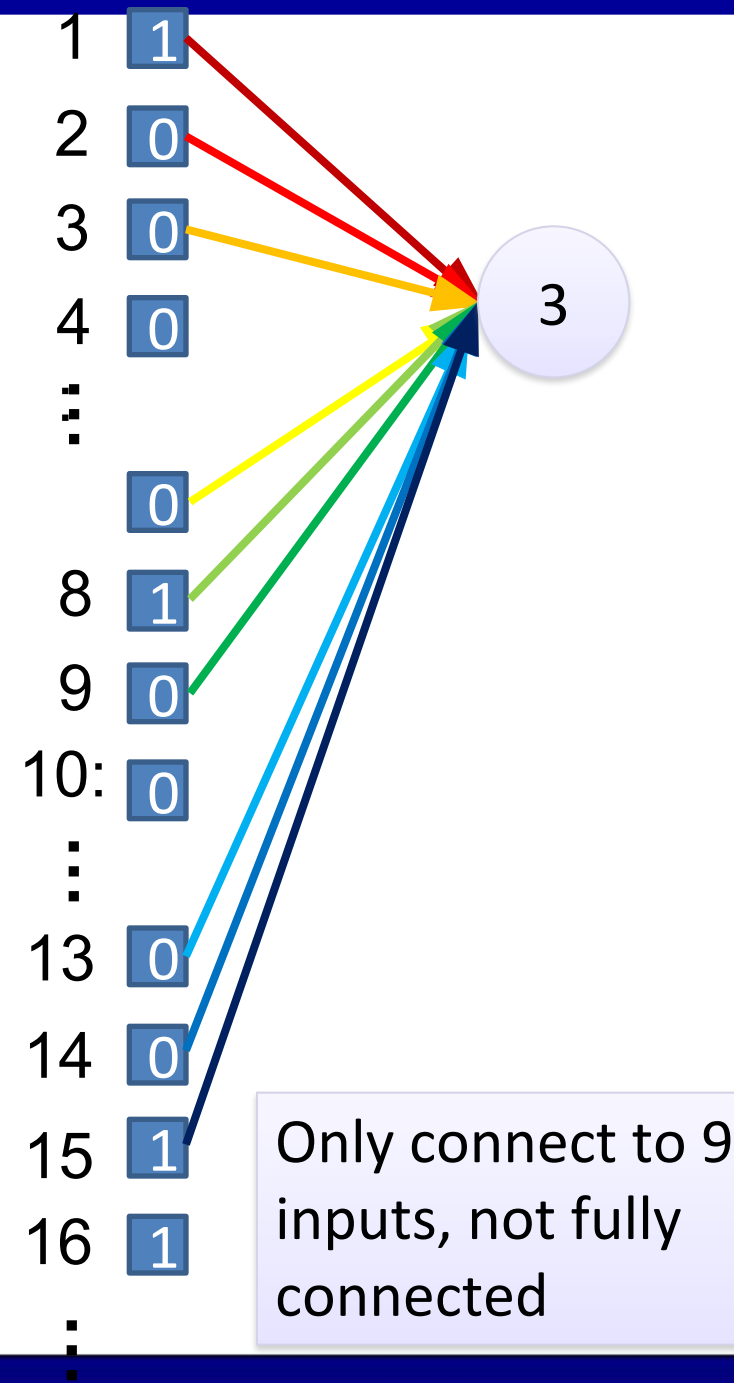


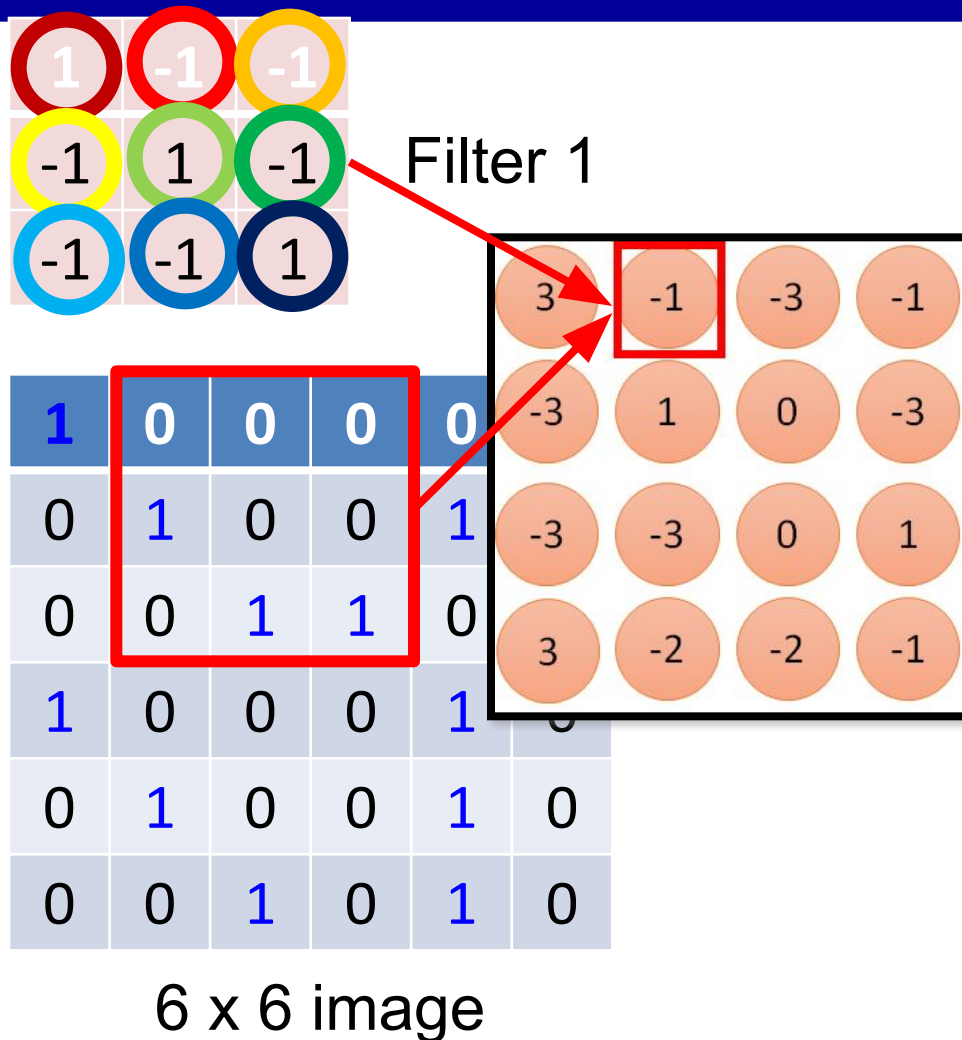
Fully-connected





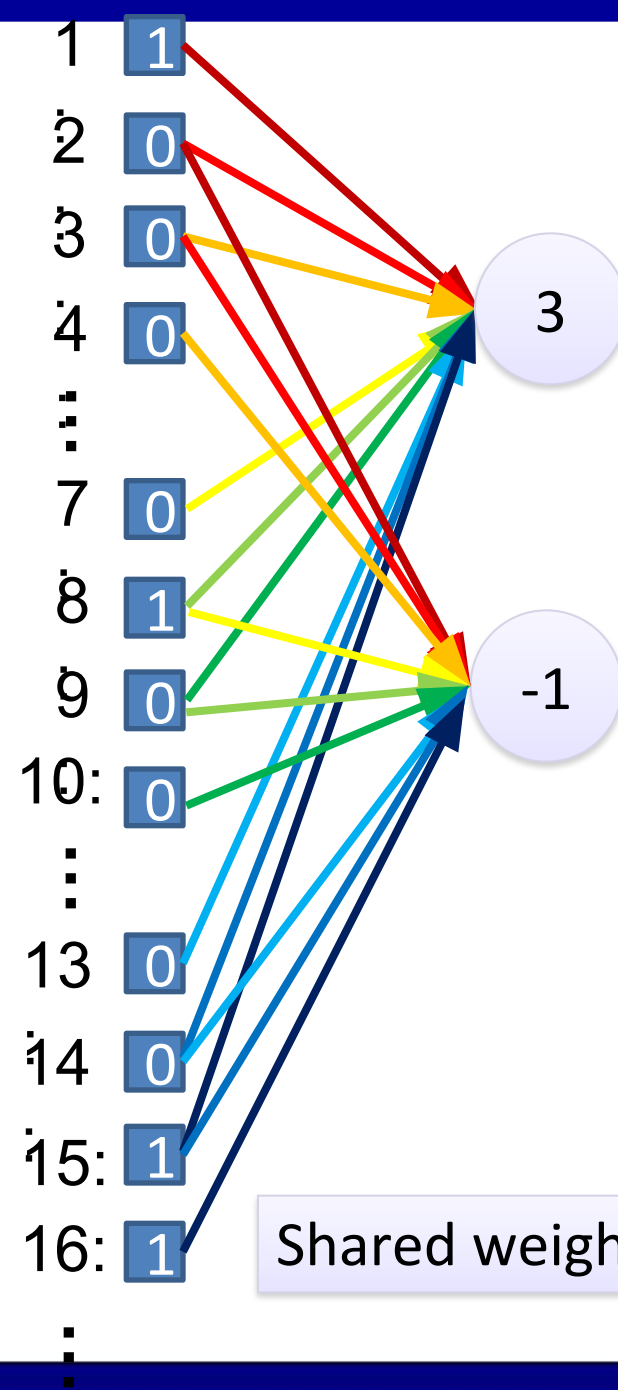
fewer parameters!





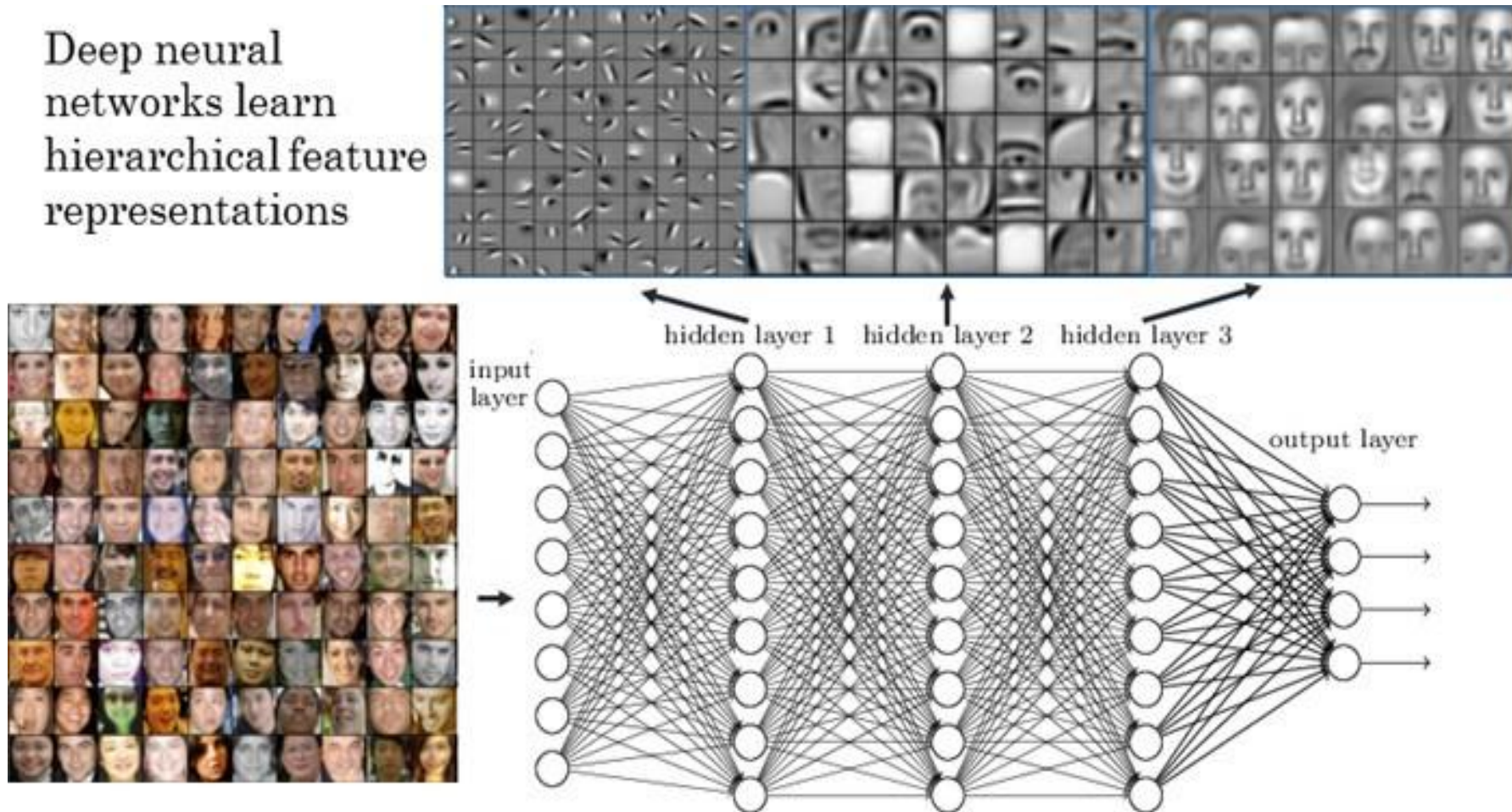
Fewer parameters

Even fewer parameters



Visualize the Results

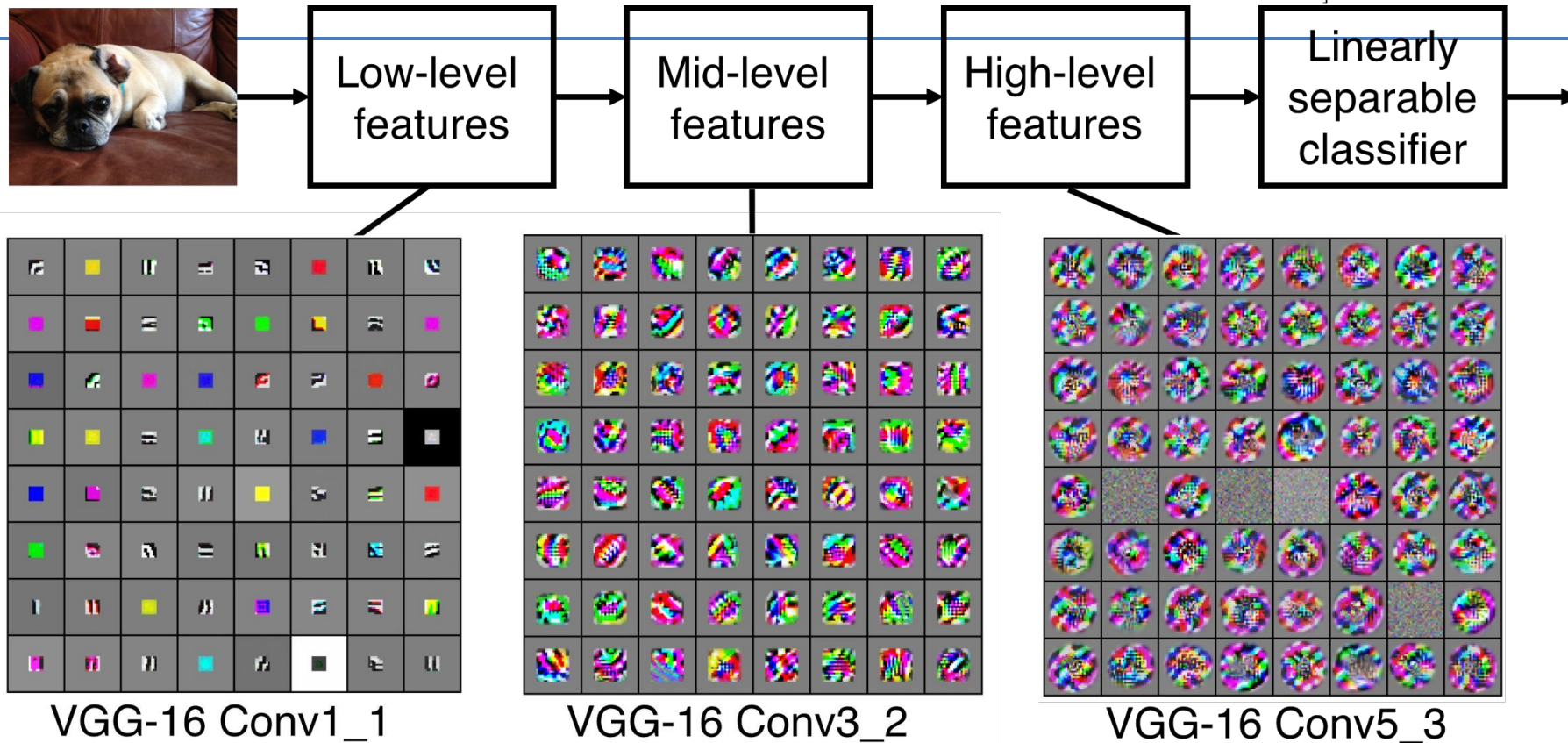
Deep neural networks learn hierarchical feature representations



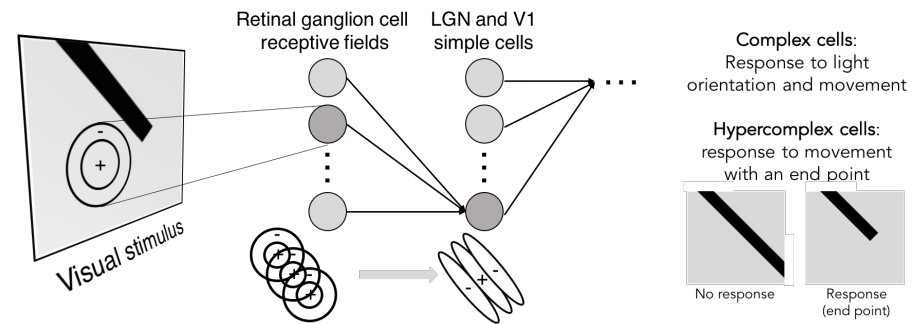
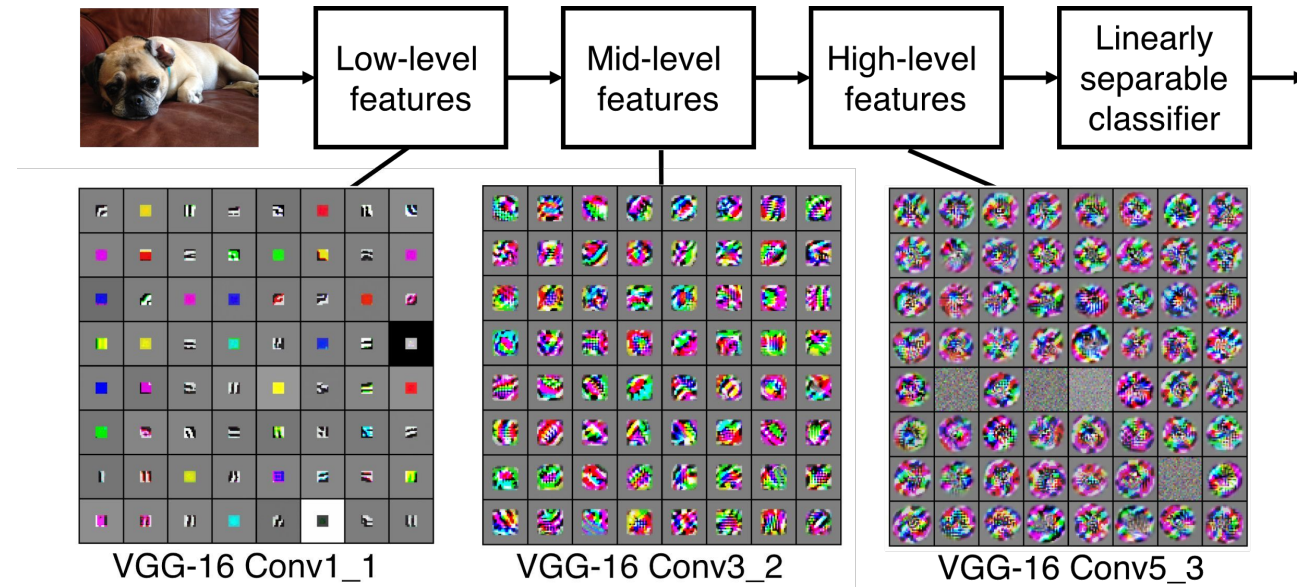
Preview

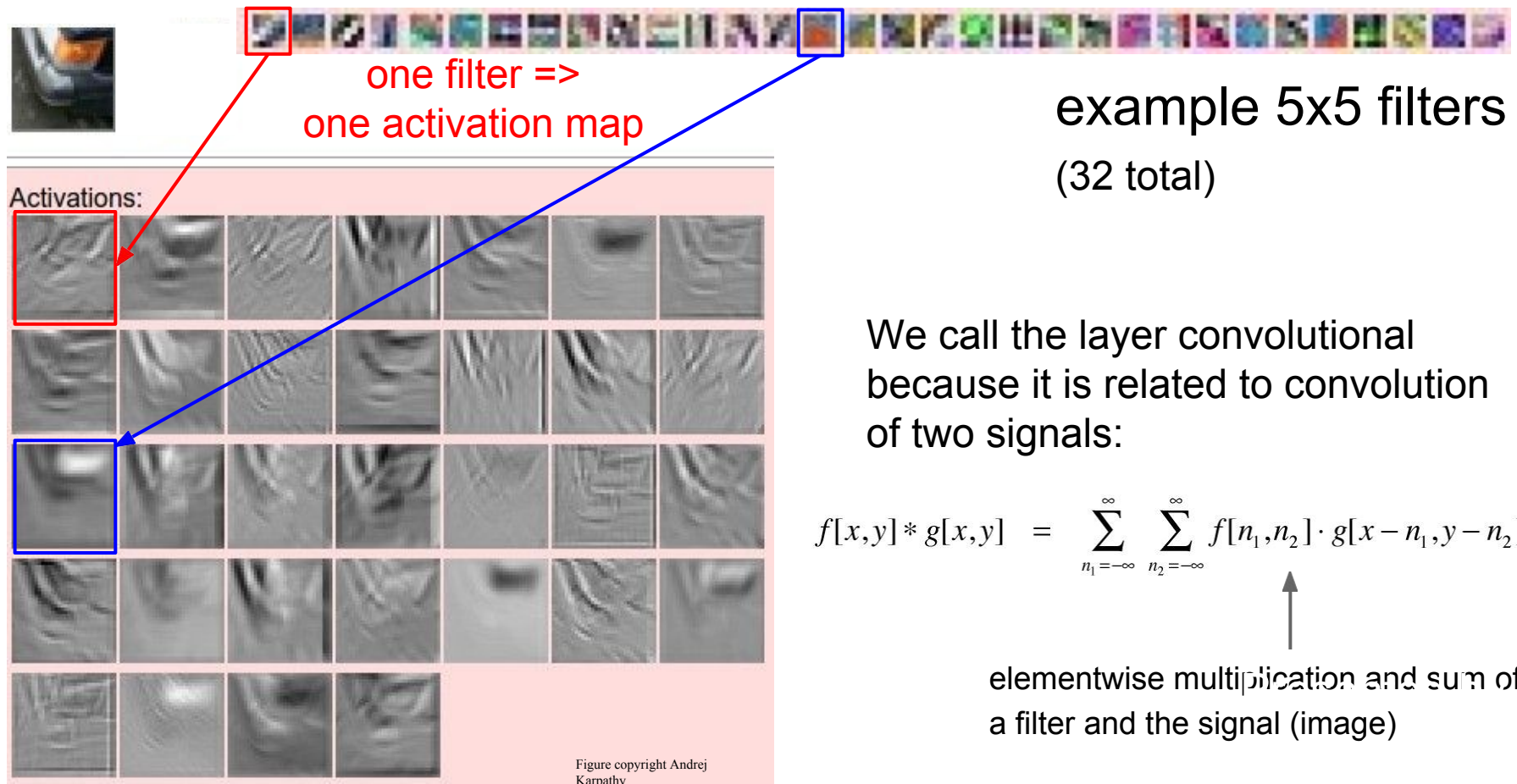
[Zeiler and Fergus 2013]

Visualization of VGG-16 by Lane McIntosh.
VGG-16 architecture from [Simonyan and Zisserman 2014].

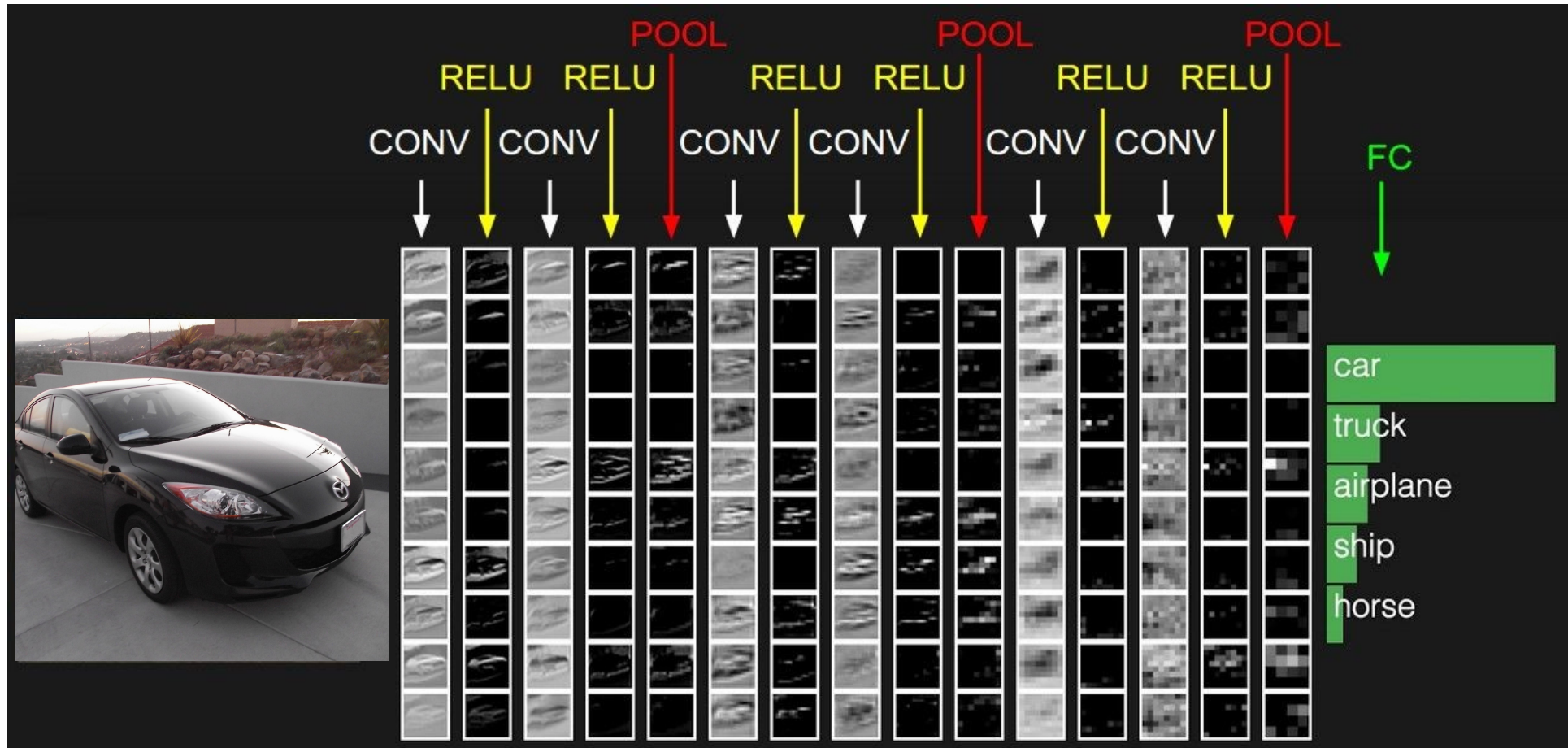


Preview

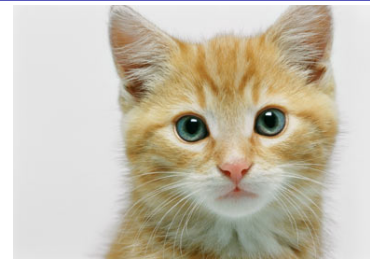




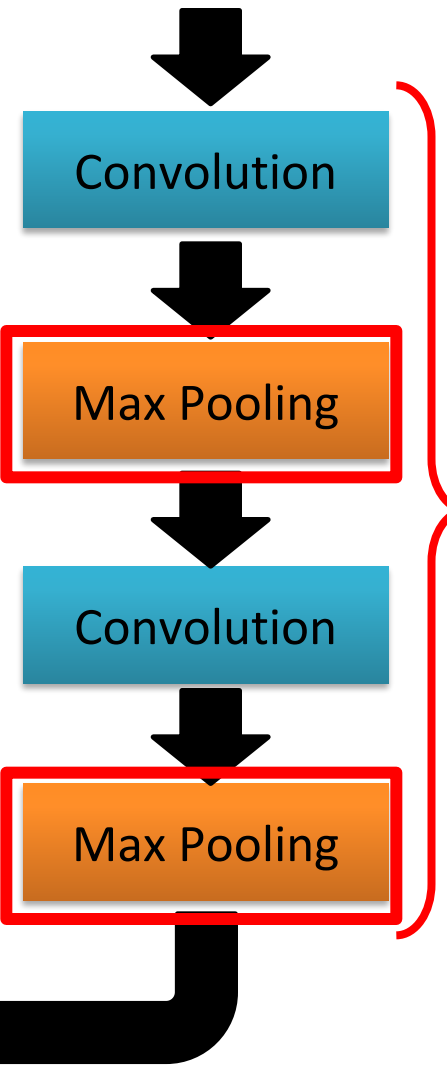
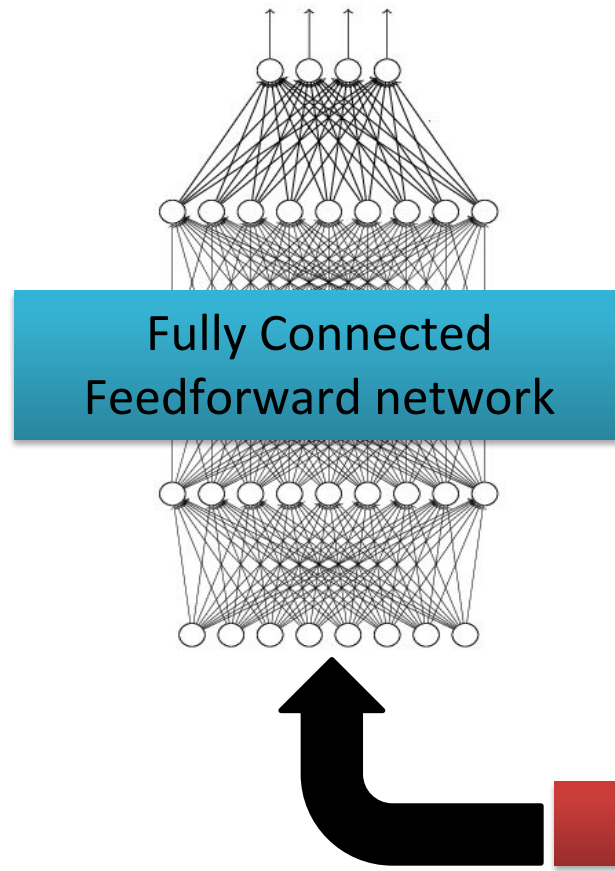
preview:



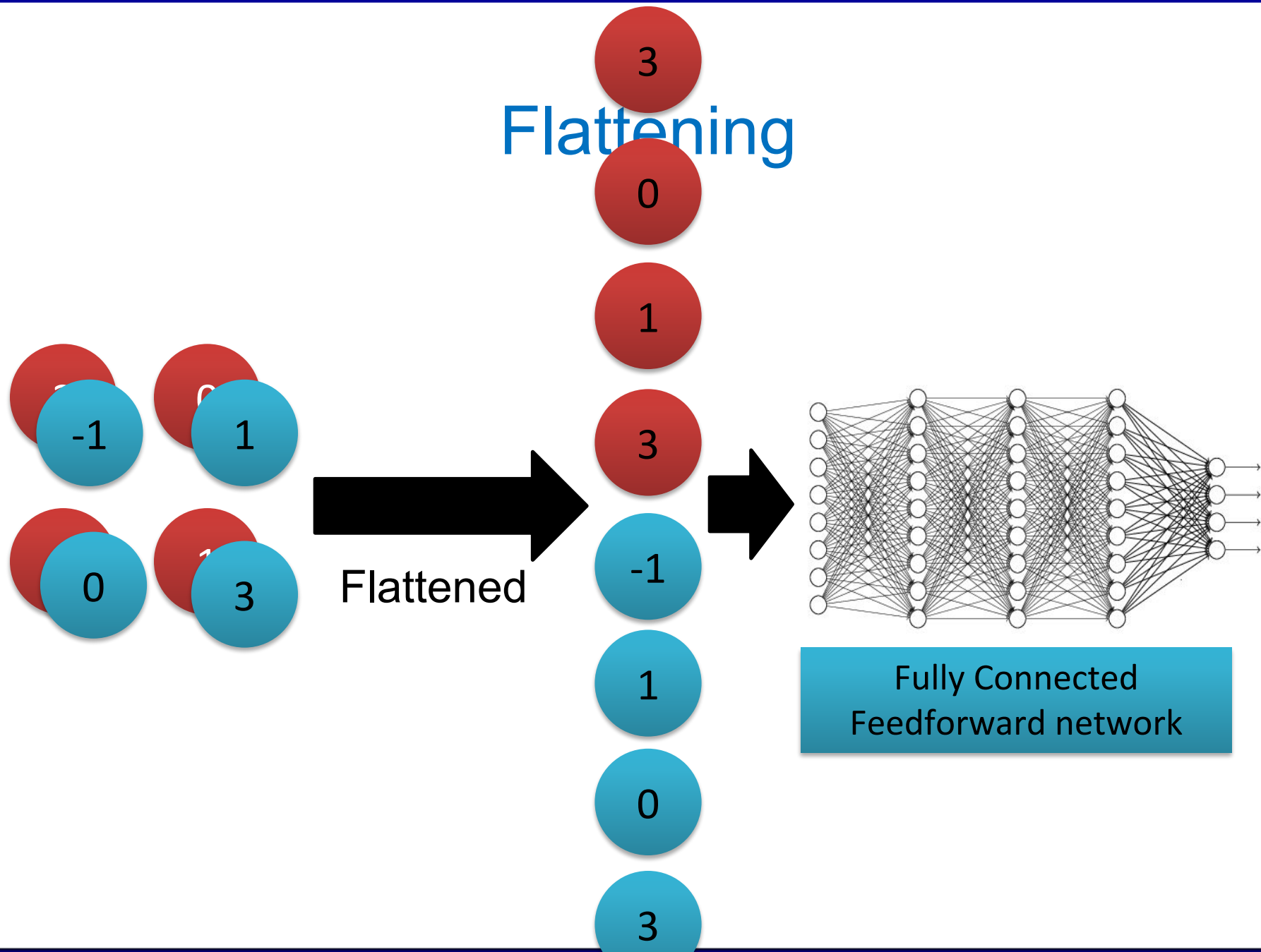
The whole



cat dog



Can repeat many times



CNN in Keras

Only modified the *network structure* and *input format (vector -> 3-D tensor)*

```
model2.add( Convolution2D( 25, 3, 3,  
                           input_shape=(28, 28, 1)) )
```

1	-1	-1	1	-1
-1	1	-1	1	-1
-1	-1	-1	1	-1

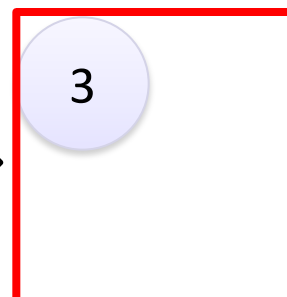
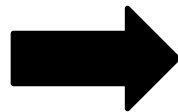
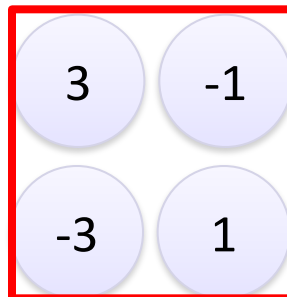
There are
25 3x3
filters.

Input_shape = (28 , 28 , 1)

28 x 28 pixels

1: black/white, 3: RGB

```
model2.add(MaxPooling2D( (2, 2) ))
```



input

Convolution

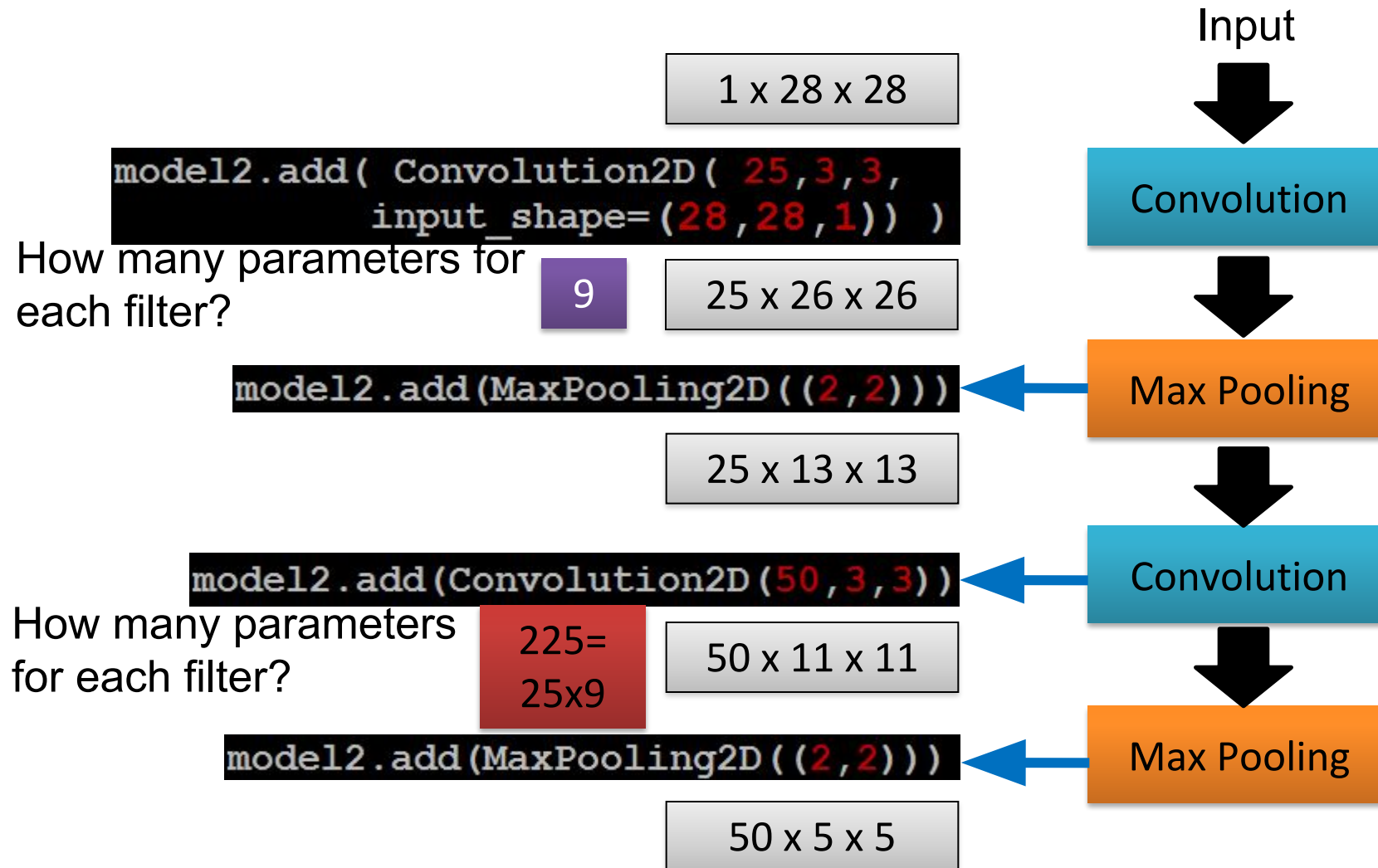
Max Pooling

Convolution

Max Pooling

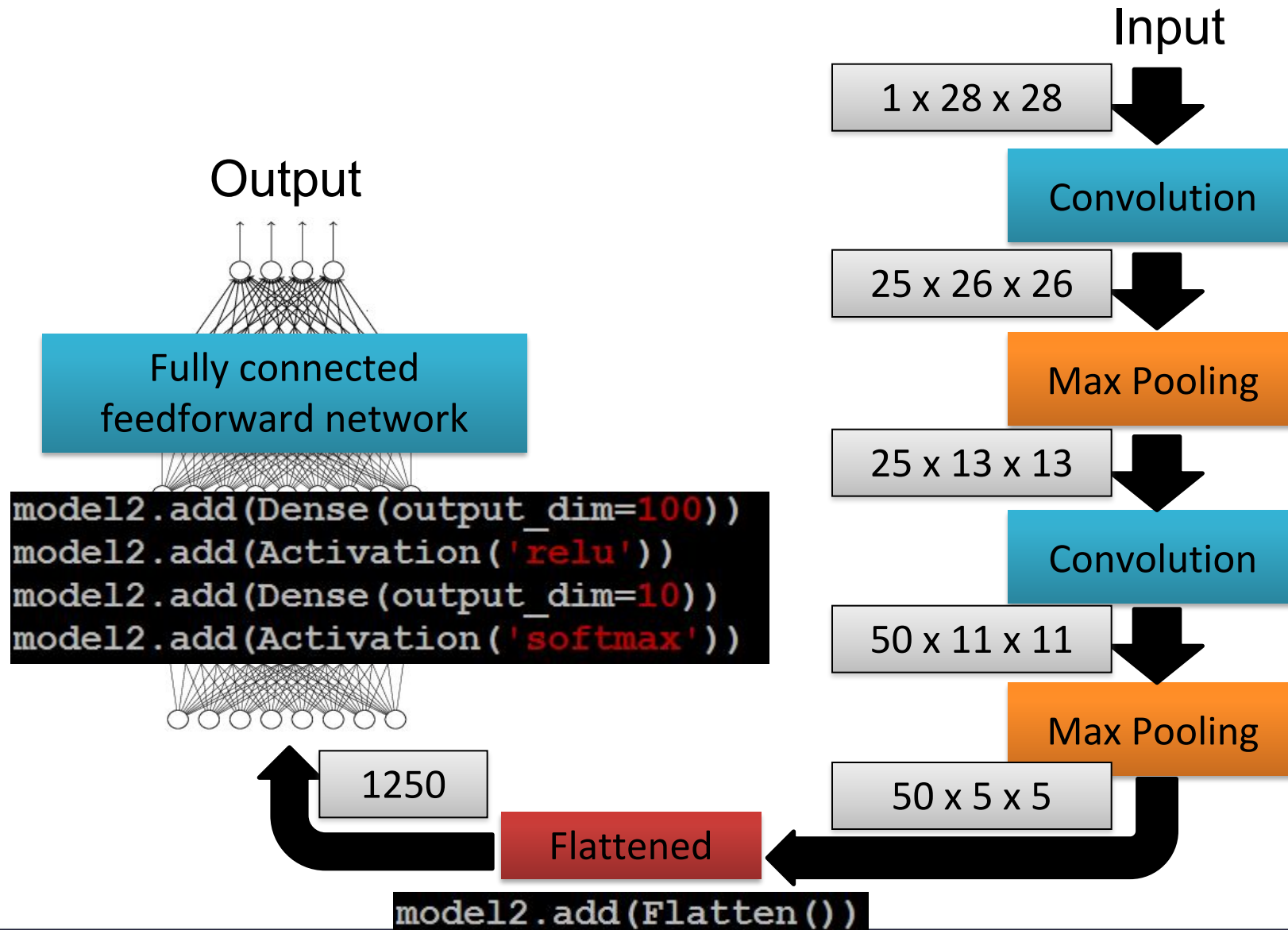
CNN in Keras

Only modified the *network structure* and *input format (vector -> 3-D array)*



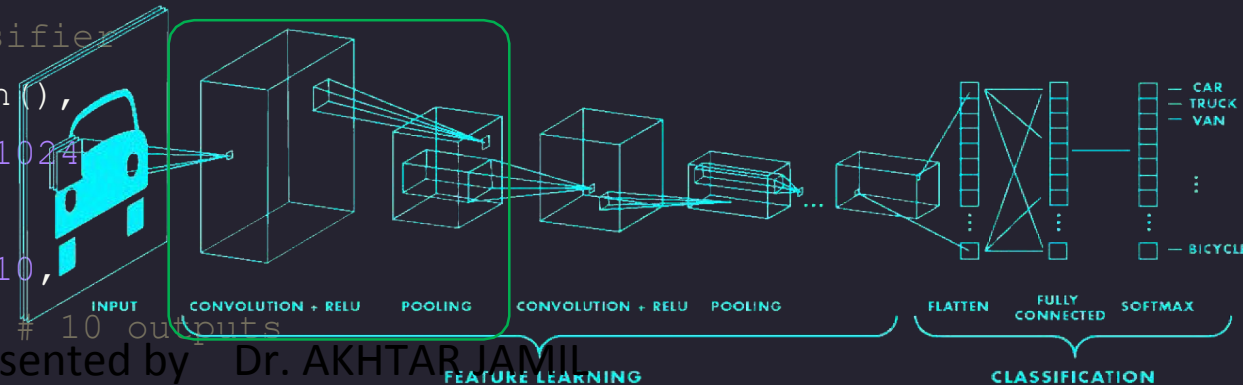
CNN in Keras

Only modified the *network structure* and *input format (vector -> 3-D array)*

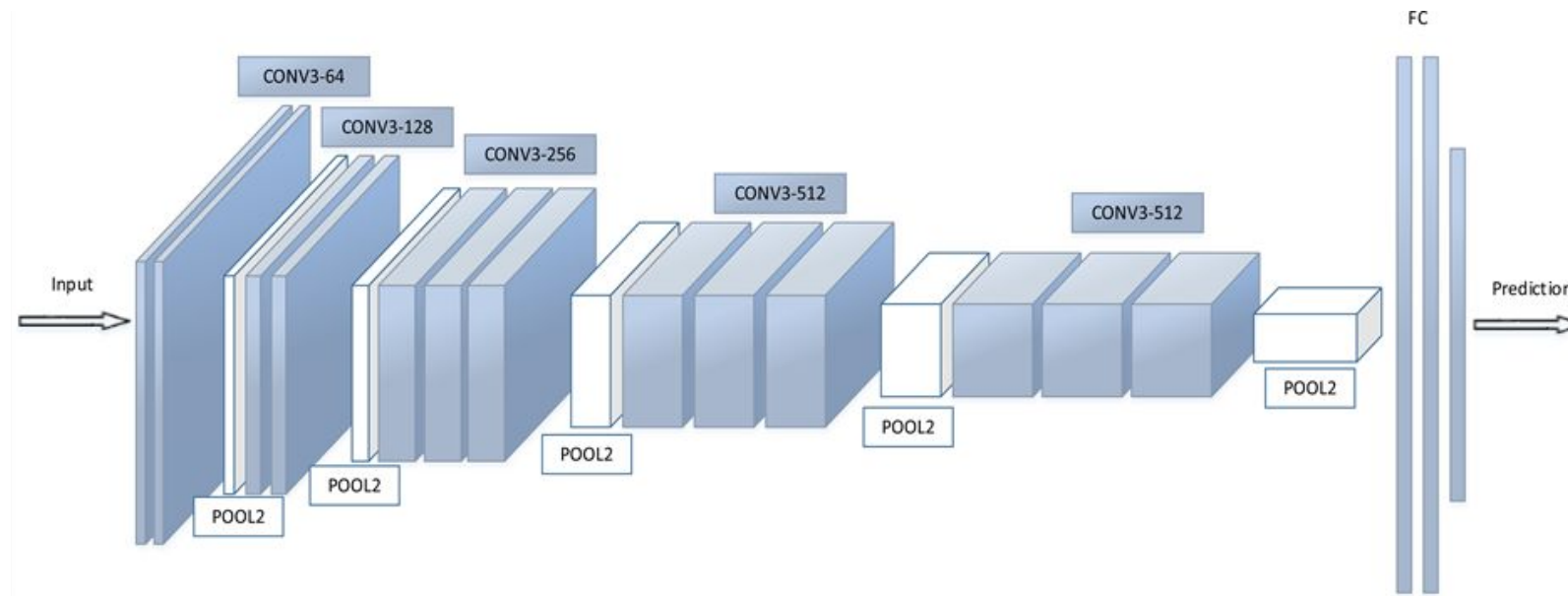


together

```
def generate_model():  
    import tensorflow as tf  
    model = tf.keras.Sequential([  
  
        # first convolutional  
        layer  
        tf.keras.layers.Conv2D(32, filter_size=3,  
                                activation='relu'), tf.keras.layers.MaxPool2D(pool_size=2,  
                                strides=2),  
  
        # second convolutional layer  
        tf.keras.layers.Conv2D(64, filter_size=3,  
                                activation='relu'), tf.keras.layers.MaxPool2D(pool_size=2,  
                                strides=2),  
  
        # fully connected classifier  
        tf.keras.layers.Flatten(),  
        tf.keras.layers.Dense(1024,  
                                activation='relu'),  
        tf.keras.layers.Dense(10,  
                                activation='softmax') # 10 outputs
```



Fully Connected Layers



Fully Connected Layers

- Regular neural network
- This layer takes an input volume from last layer of CNN and outputs an N dimensional vector.
 - Where N is the number of classes

Pooling Layer

- Pooling is used to progressively **reduce the spatial size** of the input
 - To reduce the amount of parameters and computation
 - It control overfitting (**regularization**).
 - **Invariance to small translations** of the input
- Commonly insert a Pooling layer in-between successive Conv layers

Pooling Layer

Max Pooling

29	15	28	184
0	100	70	38
12	12	7	2
12	12	45	6

2 x 2
pool size

100	184
12	45

Average Pooling

31	15	28	184
0	100	70	38
12	12	7	2
12	12	45	6

2 x 2
pool size

36	80
12	15